```
defmodule({'main': function (require, exports) {
  var Comments = require('./comments').Comments;
}});

defmodule({'comments': function (require, exports) {
  var Comments;
  Comments = (function() {
    function Comments() {}
 return Comments;
  })();

  exports.Comments = Comments;
}});
```

Each module is passed to a defmodule function. This allows the modules to be loaded as required and to have require and exports defined in their scope by being passed in as arguments. This might remind you of dependency injection from chapter 10.

AMD modules?

Another technique for defining modules is Asynchronous Module Definitions (AMD). AMD uses a `define` function that accepts a module name, its dependencies, and the module definition. In AMD, an object returned from the function contains the properties that are exported from the module. Because the setup is similar to what you just implemented, when you encounter AMD modules in the wild, you'll understand the basic premise behind them.

At this point you get a little help from Agtron. He writes tests and produces the solution shown in the following listing. It's very likely that in practice you'll use an existing library to provide modules on the client, and many of them will use techniques similar to the following listing. It's always useful to understand how things work, though, so take some time to explore the solution.

Listing 12.13. `require` and `defmodule` for the browser (lib/modules.coffee)